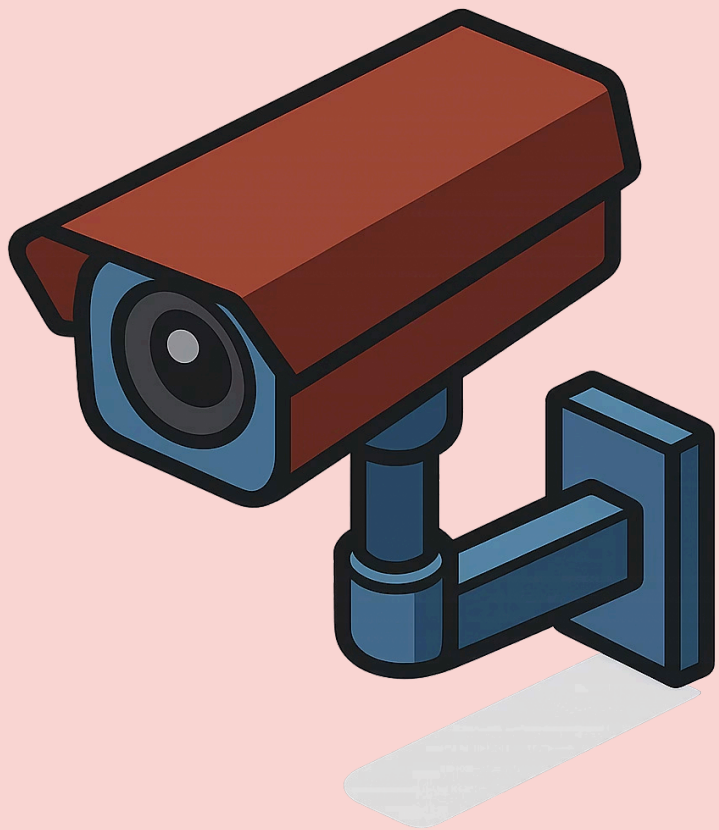




Детекция и сегментация в реальных задачах

ЛЕКЦИИ 7–9

Детекция (Object Detection) —

найти **где** объект и **что** это за объект: bounding box + класс.

Сегментация (Segmentation) —

определить **какие пиксели** принадлежат объекту:
попиксельная маска вместо рамки.

Цель курса: освоить метрики (IoU, mAP, Dice), научиться диагностировать типовые ошибки и улучшать модели системно — без шаманства.

Детекция vs сегментация: когда что выбирать

Детекция — рамки

Нужен **примерный контур** и высокая скорость. Важны объекты, их счёт и позиция в кадре. Пример: подсчёт машин, людей, товаров на полке.

Сегментация — маски

Нужна **точная форма, площадь или граница**. Важны пиксели: контакты, перекрытия, дефекты. Пример: измерение площади повреждения, медицинский анализ.

❗ Практическое правило: если важны **миллиметры и пиксели** → сегментация; если важны **объекты и их расположение** → детекция.



Bounding Box и Confidence Score

Анатомия предсказания

- **Bounding box (bbox):** прямоугольник вокруг объекта — задаётся как $(x1, y1, x2, y2)$ или (x, y, w, h)
- **Class:** категория объекта (метка класса)
- **Confidence score:** уверенность модели в наличии и принадлежности объекта к классу

Два порога в продакшене

- **Confidence threshold:** отсекаем слабые предсказания; поднимаем — меньше FP, больше FN
- **NMS IoU threshold:** управляет слиянием дублирующих рамок на один объект

❏ Оба порога — это "рабочая точка" системы, которую настраивают под бизнес-цену ошибок.

Разметка для детекции: форматы и грабли

Pascal VOC

Координаты bbox в пикселях, формат `xxyy` (x_1, y_1, x_2, y_2). XML-файлы на каждое изображение. Широко используется в классических пайплайнах.

COCO JSON

Bbox в формате `xxywh` + категории + опционально маски. Всё в одном JSON-файле. Стандарт для instance segmentation и крупных датасетов.

Типичные грабли

Перепутаны координаты или оси ($x \leftrightarrow y$), bbox выходит за границу кадра, смешаны разные конвенции форматов в одном проекте.

 **Правило №1:** всегда валидируйте разметку **визуализацией** до начала обучения. Испорченные лейблы дороже исправить после.

IoU — основная геометрия качества

Что такое IoU

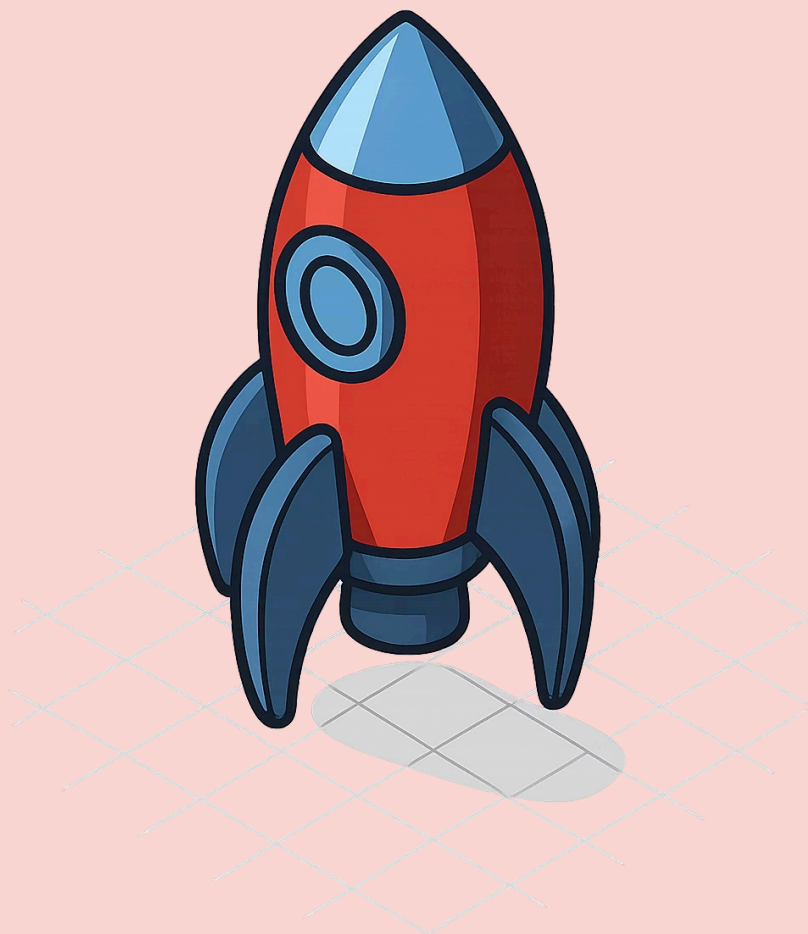
Intersection over Union — отношение площади пересечения двух прямоугольников к площади их объединения.

- IoU = **1.0** — идеальное совпадение
- IoU = **0.5** — типичный порог TP/FP
- IoU = **0.0** — рамки не пересекаются

Где используется IoU

- Оценка качества локализации
- Matching предсказаний с ground truth
- Non-Maximum Suppression (NMS)
- Порог для определения TP/FP в AP





NMS: зачем "подавлять дубликаты"

Non-Maximum Suppression (NMS) — алгоритм, который из группы перекрывающихся рамок на один объект оставляет лучшую (с наибольшим confidence), а остальные отбрасывает.

IoU threshold слишком мягкий

Много дублирующих рамок на один объект. Оператор видит несколько детекций там, где объект один.

IoU threshold слишком жёсткий

Пропуски соседних объектов — модель "съедает" рядом стоящие объекты, принимая их за дубликаты.

Правильная настройка

Подбирается на dev-наборе с учётом плотности объектов и допустимых ошибок в конкретной задаче.

Precision / Recall в детекции

1

True Positive (TP)

Объект найден: $\text{IoU} \geq \text{порога}$ и класс верный. Одно GT — не более одного TP.

2

False Positive (FP)

Ложная рамка: фон, неверный класс или слишком низкий IoU. Нагружает оператора или бизнес-логику.

3

False Negative (FN)

Пропущенный объект. Критично в задачах безопасности или медицинской диагностики.



Практически важно: в детекции FP часто "дороже" FN — ложные срабатывания создают ложную нагрузку на оператора и снижают доверие к системе. Стоимость ошибок определяет выбор рабочего порога.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

AP и mAP — главные метрики детектора

Что такое AP

Average Precision — площадь под PR-кривой для одного класса при заданном IoU-пороге. Агрегирует качество по всем порогам confidence.

Что такое mAP

mean AP — среднее значение AP по всем классам. Общий показатель качества детектора.

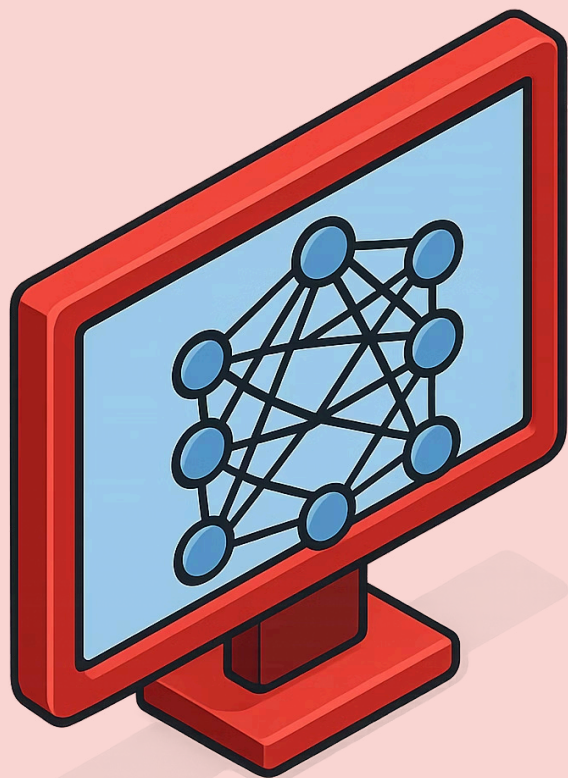
mAP@0.5

$\text{IoU} \geq 0.5$. Более мягкий порог, стандарт PASCAL VOC. Хорошо отражает, "нашли ли объект".

mAP@[.5:.95]

Среднее по IoU 0.5...0.95 с шагом 0.05. Стандарт COCO. Строже: штрафует за плохую локализацию.

❗ Для сравнения моделей всегда указывайте **какой именно mAP** используется — mAP@0.5 и mAP@[.5:.95] могут давать очень разные выводы об одной и той же модели.



YOLO в двух словах

Принцип работы

YOLO — **one-stage detector**: за один проход сети предсказывает bbox, классы и confidence одновременно для всей сетки ячеек изображения. Нет отдельного этапа region proposal.

- Выход: набор рамок + классы + confidence
- После сети: фильтрация по confidence + NMS

Ключевые параметры эксплуатации

- **Input size**: влияет на точность и скорость
- **Confidence threshold**: отсекает слабые детекции
- **NMS IoU threshold**: слияние дублей

📌 YOLO — отличный baseline, но его надо **кормить данными**. Архитектура не заменяет качество и объём обучающей выборки.

Детектор как система: пороги и рабочая точка

В продакшене модель живёт не "в среднем по метрике", а в конкретной точке пространства порогов — **operating point**.



Confidence threshold

Прямо управляет балансом FP/FN.
Поднимаем — меньше ложных срабатываний, больше пропусков.
Опускаем — наоборот.



NMS IoU threshold

Управляет дублями и соседними объектами. Влияет на плотные сцены и близко расположенные объекты.

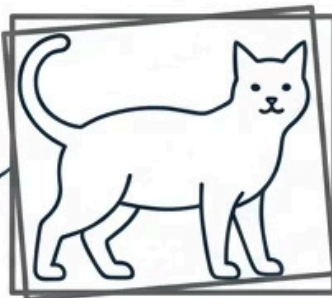


Выбор рабочей точки

Делаем на dev-выборке, исходя из **стоимости ошибок** в конкретном бизнес-контексте — не из "максимизации mAP".

ОШИБКА ЛОКАЛИЗАЦИИ

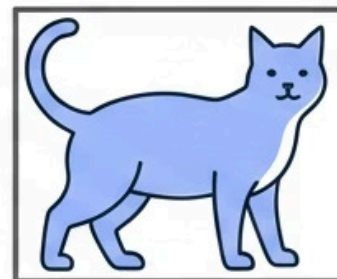
Плохая подгонка
рамки (низкий IoU).



ОШИБКА КЛАССИФИКАЦИИ

Верная рамка,
неверный класс.

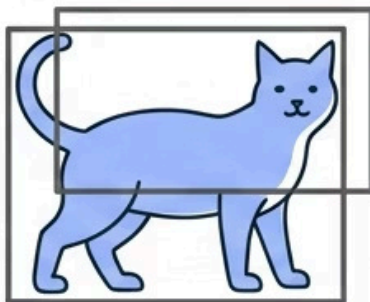
СОБАКА ✗



ДУБЛИРУЮЩИЕСЯ ДЕТЕКЦИИ

Много рамок
на один объект.

~~NMS / ПОРОГ~~



ЛОЖНЫЕ СРАБАТЫВАНИЯ ФОНА

Объект там,
где его нет.

КОТ ✗





Анализ ошибок: как делать инженерно

01

Галерея ошибок

Просматриваем FP и FN на примерах. Это обязательный ритуал — визуальный осмотр даёт интуицию, которую не заменят цифры.

02

AP по классам

Смотрим метрики отдельно для каждого класса. Редкие или похожие классы часто тянут mAP вниз.

03

По размерам объектов

Small / medium / large в COCO-стиле. Маленькие объекты — главная боль большинства детекторов.

04

По условиям съёмки

Ночь/день, разные камеры, ракурсы. Деграция в конкретных условиях указывает на доменный сдвиг.

Улучшение №1: данные и разметка

ДАННЫЕ

Главный принцип

Улучшения детектора **чаще всего начинаются с данных**, а не с замены backbone или поиска новой архитектуры. Данные — это топливо.

Практические шаги

- **Hard negatives:** добавляем фоновые примеры, которые часто дают FP в проде
- **Починка разметки:** пропуски, кривые рамки, несогласованные правила между разметчиками
- **Балансировка доменов:** разные камеры, условия освещения, ракурсы должны быть представлены в обучении

Улучшение N°2: сэмплинг и дисбаланс классов

СЭМПЛИНГ

."Дисбаланс классов существует и в детекции: редкие классы плохо представлены в батчах и "не учатся

1

Oversampling

Увеличиваем частоту показа изображений с редкими классами во время обучения.

2

Class-balanced sampling

Формируем батчи так, чтобы каждый класс встречался примерно одинаково часто.

3

Мониторинг AP per class

Оцениваем не только общий mAP, но и AP каждого класса отдельно — редкие классы легко "утонуть" в среднем.

Улучшение N°3: аугментации для детекции

АУГМЕНТАЦИИ

Стандартные рабочие

- **Scale jitter**: случайное масштабирование
- **HSV jitter**: изменение яркости/цвета
- **Blur / noise**: если в проде есть шумные камеры
- **Random crop bbox-safe**: обрезка с сохранением видимости объектов

Специфичные для детекции

- **Mosaic**: склейка 4 изображений — отлично для мелких объектов
- **MixUp**: смешивание — аккуратно, не всегда полезно
- **Copy-Paste**: вставка объектов из других сцен (требуется маски)

⚠ **Риск**: аугментации могут испортить лейблы — объект ушёл за кадр или оказался почти полностью обрезан. Следите за **bbox visibility threshold** и валидируйте после аугментации.

Улучшение №4: гиперпараметры и размер входа

ГИПЕРПАРАМЕТРЫ



Input size

Большой размер входа → лучше для мелких объектов, но выше требования к памяти и ниже скорость инференса.



LR / Scheduler / Epochs

Классическая отладка обучения: warmup, cosine decay, ранняя остановка. Ничего нового, но критично для сходимости.



Ablation study

Меняем **по одной вещи за раз**.
Иначе при улучшении метрики не поймём, что именно помогло.

Переход к сегментации: что она выдаёт

Semantic Segmentation

Каждый пиксель получает метку класса: фон, дорога, автомобиль и т.д. Экземпляры одного класса не различаются.

Instance Segmentation

Отдельная маска для каждого экземпляра объекта. Два человека рядом — две разные маски. Пример: Mask R-CNN.

Когда нужна сегментация

Вдох недостаточно: измерение площади дефекта, точная граница объекта, анализ контактов и перекрытий.



Метрики сегментации: IoU и Dice

Pixel IoU

Тот же принцип, что для bbox, но вычисляется по пикселям маски:

$$\text{IoU} = \text{TP} / (\text{TP} + \text{FP} + \text{FN})$$

где TP — совпавшие пиксели, FP — лишние, FN — пропущенные.

Dice (F1 по пикселям)

$$\text{Dice} = 2 \cdot \text{TP} / (2 \cdot \text{TP} + \text{FP} + \text{FN})$$

Математически эквивалентен F1-score на уровне пикселей.
Более чувствителен к малым объектам.

i IoU vs Dice: Dice сильнее штрафует за дисбаланс. При тонких границах рекомендуется также смотреть **boundary IoU** — отдельно оцениваем качество краёв маски, а не только площадное совпадение.

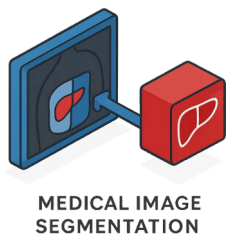
Mean IoU (mIoU)

Среднее IoU по классам.
Стандарт для semantic segmentation.

Dice per class

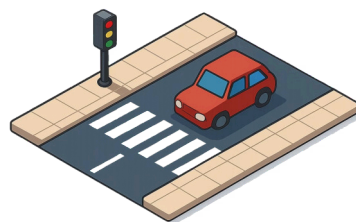
Особенно полезен в медицинских задачах с сильным дисбалансом фон/объект.

Архитектуры сегментации: выбор в реальности



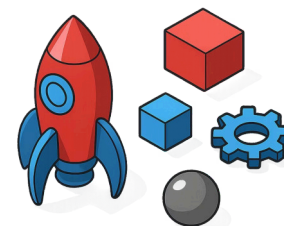
U-Net

Классика encoder-decoder с skip connections. Сильна на **малых датасетах** — особенно в медицине и промышленной дефектоскопии. Просто обучать.



DeepLab

Сильный семантический контекст за счёт dilated convolutions и ASPP. Хорош для **сцен с крупными объектами** и сложным фоном.



Mask R-CNN

Instance segmentation: bbox + маска на каждый экземпляр. Мощно, но требует **сложной разметки** и больше ресурсов.

Выбор в реальности определяется треугольником: **качество** ↔ **скорость** ↔ **сложность разметки**.

Итог: как строить улучшения системно

1



Метрики

IoU/NMS → AP/mAP для детекции; pixel IoU / Dice для сегментации

2



Диагностика ошибок

Локализация / класс / дубли / фон — раскладываем по корзинам, не смотрим только mAP

3



Данные и разметка

Hard negatives, починка лейблов, балансировка доменов

4



Сэмплинг и аугментации

Oversampling редких классов, Mosaic, Copy-Paste, bbox-safe crop

5



Гиперпараметры и модель

Input size, LR, пороги — меняем по одному (ablation). Модель — последний шаг.



Дальше в курсе: OCR (лекция 12) → Foundation models (лекция 14) → Деплой и ONNX (лекция 15)